МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ
Кафедра многопроцессорных систем и сетей

НАИМЕНОВАНИЕ ОТЧЕТА

Курсовой проект

Хоронеко Дмитрия Николаевича
обучающегося 3 курса специальности
«информатика»

Научный руководитель:
Старший преподаватель кафедры
многопроцессорных систем и сетей
Кондратьева Ольга Михайловна

Минск, 2025

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 3
ГЛАВА 1 Анализ предметной области и выбор технологий 4
1.1 Анализ существующих решений 4
1.2 Постановка задачи 4
1.3 Обоснование выбора технологий 6
ГЛАВА 2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ 9
2.1 Архитектура веб-приложения 9
2.2 Проектирование базы данных 10
2.3 Проектирование пользовательского интерфейса 11
ГЛАВА 3 ПРОЕКТИРОВАНИЕ СИСТЕМЫ 13
3.1 Разработка backend на Django 13
3.2 Реализация моделей данных 13
3.3 Разработка REST API 14
3.4 Разработка frontend на React 15
3.5 Реализация системы поиска и фильтрации 16
3.6 Интеграция с системой учета 1С Предприятие 18
3.7 Контейнеризация приложения 20
ЗАКЛЮЧЕНИЕ 21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 22

ВВЕДЕНИЕ

Современный рынок зоотоваров характеризуется высокими темпами роста и широким ассортиментом продукции. Владельцы домашних животных сталкиваются с широким выбором кормов и товаров для питомцев, что часто приводит к затруднениям в выборе оптимального продукта. Многие потребители, особенно неопытные, не обладают достаточными знаниями о составе кормов, их влиянии на здоровье животных и критериях выбора качественной продукции. Поэтому появилась необходимость в создании удобного каталога, который будет содержать всю необходимую информацию для выбора подходящих товаров.

Актуальность данной работы обусловлена несколькими факторами:

– ростом числа владельцев домашних животных и увеличением спроса на качественные зоотовары;

– недостатком удобных информационных ресурсов для просмотра и сравнения товаров;

– необходимостью структурированного представления информации о составе, характеристиках и ценах товаров;

– потребностью в эффективных инструментах поиска и фильтрации при большом ассортименте продукции.

Практическая значимость работы состоит в создании функционального веб-приложения, которое может быть использовано зоомагазинами для предоставления клиентам удобного доступа к информации о товарах. Система позволяет пользователям быстро находить необходимые товары, сравнивать их характеристики и принимать обоснованные решения о покупке.

В работе используются следующие технологии:

– Django – высокоуровневый веб-фреймворк на языке Python для разработки backend;

– Django REST Framework – инструментарий для создания Web API;

– React – JavaScript-библиотека для создания пользовательских интерфейсов;

– PostgreSQL – объектно-реляционная система управления базами данных.

ГЛАВА 1 Анализ предметной области и выбор технологий

1.1 Анализ существующих решений

Современный рынок веб-каталогов товаров для животных представлен различными решениями, от простых статических страниц до сложных интернет-магазинов с полным функционалом покупки. Проведем анализ основных типов существующих решений.

Полнофункциональные интернет-магазины включают не только каталог, но и корзину, систему оплаты, личный кабинет пользователя. Такие решения обладают избыточной функциональностью для задачи информационного представления товаров. Их недостатки:

– высокая сложность разработки и поддержки,
– необходимость обеспечения безопасности платежей,
– требование регистрации пользователей,
– перегруженность интерфейса дополнительными элементами.

Информационные каталоги представляют решение, фокусирующееся на представлении товаров без функций покупки. Именно такой подход выбран для данного проекта. Основные требования к информационному каталогу:

– структурированное представление товаров по категориям,
– детальная информация о составе и характеристиках,
– эффективная система поиска и фильтрации,
– адаптивный дизайн для различных устройств,
– высокая производительность при большом объеме данных.

Анализ существующих решений показывает следующие проблемы:

– недостаточная детализация характеристик товаров,
– отсутствие информации о составе кормов,
– слабая структуризация по типам животных, возрасту,
– низкая скорость работы при большом ассортименте.

Для решения выявленных проблем в разрабатываемой системе будут реализованы:

– детальные карточки товаров с полным описанием состава,
– многоуровневая система категорий,
– гибкая фильтрация по множеству параметров,
– быстрый полнотекстовый поиск,
– адаптивный интерфейс с оптимизацией для мобильных устройств.

1.2 Постановка задачи

На основе проведенного анализа формулируется следующая постановка задачи.

Необходимо разработать динамический веб-каталог зоомагазина, который предоставляет пользователям возможность просмотра информации о товарах для животных без функции оформления покупки. Система должна быть исключительно информационным ресурсом. Функциональные требования к системе:

– Каталог товаров должен обеспечивать:

1) просмотр списка товаров с разбиением по категориям (корма, аксессуары, игрушки, средства гигиены и т.д.);
2) детальную информацию о каждом товаре, включающую название, описание, состав, вес, производителя, страну производства и цену;
3) изображения товаров с возможностью увеличения;
4) структурирование по типам животных (собаки, кошки, грызуны, птицы и т.д.);
5) дополнительную классификацию по возрасту животного и типу корма.

– Система поиска должна предоставлять:

1) полнотекстовый поиск по названию товара;
2) поиск по ключевым словам в описании и составе;
3) отображение результатов поиска с релевантной сортировкой.

– Система фильтрации должна позволять:

1) фильтрацию по ценовому диапазону;
2) фильтрацию по производителю (бренду);
3) фильтрацию по типу товара;
4) фильтрацию по типу специальным характеристикам (например, гипоаллергенный, беззерновой корм);
5) фильтрацию по специальным характеристикам (например, гипоаллергенный, беззерновой корм);
6) комбинирование нескольких фильтров одновременно.

– Пользовательский интерфейс должен обеспечивать:

1) адаптивный дизайн для корректного отображения на мобильных устройствах, планшетах и десктопах;
2) интуитивную навигацию по категориям;
3) визуально привлекательные карточки товаров;
4) быструю загрузку страниц;
5) доступность без регистрации и авторизации.

– Нефункциональные требования:

1) время загрузки страницы каталога не должно превышать 2 секунд;

2) система должна корректно работать в современных браузерах (Chrome, Firefox, Safari, Edge);

3) интерфейс должен поддерживать разрешения экрана от 320px до 4K;

4) код должен быть структурирован и документирован для возможности дальнейшего развития.

– Ограничения проекта:

1) система не включает функционал корзины и оформления заказов;

2) отсутствует система авторизации и личных кабинетов;

3) не предусмотрена интеграция с платежными системами;

4) не реализуется система отзывов и рейтингов.

1.3 Обоснование выбора технологий

Для реализации проекта выбран стек технологий Django (backend) + React (frontend) + PostgreSQL (база данных). Рассмотрим обоснование каждого выбора. Django выбран в качестве backend-фреймворка так как это высокоуровневый веб-фреймворк на языке Python, который следует принципу "batteries included" (включены батарейки), предоставляя разработчику готовые решения для типичных задач веб-разработки. Преимущества Django:

– встроенная ORM (Object-Relational Mapping) обеспечивает удобную работу с базой данных без необходимости написания SQL-запросов,

– автоматически генерируемая панель администрирования позволяет управлять данными без разработки отдельного интерфейса,

– встроенная система безопасности защищает от распространенных уязвимостей (SQL-инъекции, XSS, CSRF),

– миграции базы данных упрощают управление изменениями схемы,

– обширная экосистема пакетов расширяет функциональность,

– отличная документация и большое сообщество разработчиков,

– масштабируемость и производительность для проектов любого размера.

Для данного проекта особенно важны встроенная ORM для работы с товарами и категориями, а также админ-панель для удобного добавления и редактирования товаров.

Django REST Framework (DRF) используется для создания REST API:

– предоставляет удобные классы для создания API endpoints,

– включает сериализаторы для преобразования моделей в JSON,

– поддерживает различные методы аутентификации,

– предоставляет browsable API для тестирования,

– включает систему фильтрации и пагинации.

React выбран для frontend-разработки так как это легкая и удобная JavaScript-библиотека для создания пользовательских интерфейсов. Преимущества React:

– компонентный подход позволяет создавать переиспользуемые элементы интерфейса,

– виртуальный DOM обеспечивает высокую производительность при обновлении интерфейса,

– однонаправленный поток данных упрощает понимание и отладку приложения,

– богатая экосистема библиотек (React Router, Axios и др.),

– возможность создания SPA (Single Page Application) для плавной навигации без перезагрузки страниц,

– активное развитие и поддержка большого сообщества,

– отличные инструменты разработки (React DevTools).

Для проекта каталога React особенно подходит благодаря:

– возможности создания интерактивных фильтров без перезагрузки страницы,

– эффективному обновлению списка товаров при изменении фильтров,

– удобному управлению состоянием приложения,

– простоте создания адаптивных компонентов.

В качестве системы управления базами данных была выбрана PostgreSQL.

Преимущества PostgreSQL:

– надежность и стабильность работы,

– поддержка сложных запросов и индексов,

– расширенные возможности полнотекстового поиска,

– поддержка JSON для хранения сложных структур данных,

– ACID-совместимость обеспечивает целостность данных,

– открытый исходный код и бесплатное использование,

– отличная интеграция с Django,

– масштабируемость для больших объемов данных.

Для каталога зоомагазина PostgreSQL предоставляет:

– эффективное хранение структурированных данных о товарах,

– быстрый полнотекстовый поиск по названиям и описаниям,

– поддержку сложных запросов с фильтрацией,

– возможность хранения JSON-полей для гибких характеристик товаров.

Дополнительные технологии и библиотеки:

Axios – HTTP-клиент для выполнения запросов к API из React-приложения. Преимущества:

– Promise-based API для удобной работы с асинхронными запросами,

– автоматическое преобразование JSON,

– возможность перехвата запросов и ответов,

– поддержка отмены запросов.

React Router – библиотека для маршрутизации в React-приложении. Позволяет:

– создавать различные страницы (список товаров, детали товара),

– управлять историей браузера,

– передавать параметры через URL.

Django CORS Headers – пакет для настройки Cross-Origin Resource Sharing, необходимый для взаимодействия React-приложения с Django API.

Django Filter – библиотека для создания фильтров в Django REST Framework, упрощающая реализацию фильтрации товаров.

Выбранный стек технологий обеспечивает:

– разделение frontend и backend для независимой разработки и развертывания,

– современный подход к разработке веб-приложений,

– высокую производительность и масштабируемость,

– удобство поддержки и развития проекта,

– использование проверенных и надежных инструментов.

ГЛАВА 2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Архитектура веб-приложения

Разрабатываемая система использует архитектуру клиент-сервер с разделением на frontend и backend компоненты. Такой подход обеспечивает независимую разработку, тестирование и развертывание частей системы. Общая архитектура системы включает следующие уровни:

Уровень представления (Frontend):

– React-приложение, выполняющееся в браузере пользователя,

– отвечает за отображение интерфейса и взаимодействие с пользователем,

– отправляет HTTP-запросы к backend API,

– обрабатывает пользовательский ввод (поиск, фильтры),

– управляет клиентским состоянием приложения.

Уровень бизнес-логики (Backend):

– Django-приложение, работающее на сервере,

– предоставляет REST API для доступа к данным,

– обрабатывает запросы фильтрации и поиска,

– выполняет валидацию данных,

– управляет взаимодействием с базой данных.

Уровень данных:

– PostgreSQL база данных,

– хранит информацию о товарах, категориях, производителях,

– обеспечивает целостность и согласованность данных.

Взаимодействие компонентов:

– Пользователь взаимодействует с React-приложением через браузер. При необходимости получения данных (например, список товаров или детали конкретного товара) React отправляет HTTP-запрос к Django REST API. Django обрабатывает запрос, выполняет необходимые операции с базой данных через ORM и возвращает данные в формате JSON. React получает ответ и обновляет интерфейс для пользователя.

Преимущества выбранной архитектуры:

– разделение ответственности между компонентами,

– возможность независимого масштабирования frontend и backend,

– упрощение тестирования каждого уровня отдельно,

– возможность использования одного API для разных клиентов (веб, мобильные приложения),

– четкие интерфейсы взаимодействия между компонентами.

Backend организован в виде Django-проекта со следующими приложениями:

– zoo_catalog – основной проект, содержащий настройки,

– products – приложение для работы с товарами и категориями,

– api – приложение для REST API endpoints.

Frontend организован по компонентному принципу:

– components – переиспользуемые компоненты (карточка товара, фильтры, поиск),

– pages – страницы приложения (главная, каталог, детали товара),

– services – модули для взаимодействия с API,

– utils – вспомогательные функции,

– styles – стили приложения.

Просмотр каталога:

1. Пользователь открывает страницу каталога;
2. React отправляет GET-запрос к /api/products/;
3. Django возвращает список товаров с пагинацией;
4. React отображает товары в виде карточек.

Применение фильтров:

1. Пользователь выбирает параметры фильтрации;
2. React формирует GET-запрос с параметрами (?price_min=100&brand=Royal Canin);
3. Django применяет фильтры через Django Filter;
4. Django возвращает отфильтрованный список товаров;
5. React обновляет отображение без перезагрузки страницы.

Поиск товаров:

1. Пользователь вводит поисковый запрос;
2. React отправляет GET-запрос с параметром search;
3. Django выполняет полнотекстовый поиск в PostgreSQL;
4. Django возвращает релевантные результаты;
5. React отображает найденные товары.

Просмотр деталей товара:

1. Пользователь кликает на товар;
2. React Router переходит на страницу /product/:id;
3. React отправляет GET-запрос к /api/products/:id/;
4. Django возвращает детальную информацию о товаре;
5. React отображает страницу с полной информацией.

2.2 Проектирование базы данных

База данных разработана с учетом требований к хранению информации о товарах зоомагазина. Схема базы данных включает следующие основные таблицы:

Таблица "Category" (Категории):

– Хранит иерархическую структуру категорий товаров.

Таблица "Brand" (Производители):

– Хранит информацию о производителях товаров.

Таблица "AnimalType" (Типы животных):

– Классификация по видам животных.

Таблица "Product" (Товары):

– Основная таблица с информацией о товарах.

Таблица "ProductImage" (Изображения товаров):

– Дополнительные изображения для товаров.

Связи между таблицами:

– Category имеет самосвязь (parent_id) для создания иерархии категорий (например, Корма > Корма для собак > Сухие корма для собак).

– Product связан с Category отношением многие-к-одному – каждый товар принадлежит одной категории, но в категории может быть много товаров.

– Product связан с Brand отношением многие-к-одному – каждый товар имеет одного производителя, но производитель может выпускать много товаров.

– Product связан с AnimalType отношением многие-к-одному – каждый товар предназначен для одного типа животных.

– ProductImage связан с Product отношением многие-к-одному – один товар может иметь несколько изображений.

Индексы для оптимизации:

– индекс на Product.category_id для быстрой выборки товаров по категории;

– индекс на Product.brand_id для фильтрации по производителю;

– индекс на Product.animal_type_id для фильтрации по типу животного;

– индекс на Product.price для сортировки и фильтрации по цене;

– полнотекстовый индекс на Product.name и Product.description для поиска;

– уникальный индекс на Product.slug для быстрого поиска по URL.

Нормализация базы данных:

– База данных спроектирована в соответствии с третьей нормальной формой (3NF)

2.3 Проектирование пользовательского интерфейса

Пользовательский интерфейс проектируется с учетом принципов удобства использования (usability) и современных стандартов веб-дизайна.

Основные страницы приложения:

1. Главная страница:

Содержит:

– шапку сайта с логотипом и поиском;

– навигационное меню с основными категориями;

– баннеры с популярными категориями товаров;
– подборки товаров (новинки, популярные);
– футер с контактной информацией.

2. Страница каталога:
Основная страница для просмотра товаров.
страницы.
Фильтры:
– ценовой диапазон (слайдер);
– выбор категории (древовидный список);
– выбор производителя (чекбоксы);
– тип животного (кнопки с иконками);
– возрастная группа (радиокнопки);
– специальные характеристики (чекбоксы);
– кнопка "Сбросить фильтры".
Отображение товаров:
– карточки товаров в виде сетки;
– каждая карточка содержит: изображение, название, бренд, вес, цену;
– при наведении появляется краткое описание;
– клик на карточку открывает страницу товара.
3. Страница товара:
Детальная информация о конкретном товаре.
Структура:
– крупное изображение товара с галереей дополнительных фото;
– название и бренд;
– цена и вес;
– характеристики (тип животного, возраст, особенности);
– полное описание;
– состав товара;
Принципы проектирования интерфейса:
– минимализм – только необходимые элементы;
– согласованность – единый стиль всех компонентов;
– обратная связь – визуальные индикаторы действий пользователя;
– доступность – контрастные цвета, читаемые шрифты;
– эффективность – быстрый доступ к основным функциям.

ГЛАВА 3 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

3.1 Разработка backend на Django

Backend система разработана с использованием Django 6.0 и организована в виде модульной структуры. Основной проект zoo_catalog содержит настройки приложения, а функциональность разделена на отдельные приложения.

Настройка проекта выполнена в файле settings.py, где определены следующие ключевые параметры:

В списке установленных приложений (INSTALLED_APPS) добавлены сторонние библиотеки rest_framework для создания API, corsheaders для поддержки кросс-доменных запросов, django_filters для фильтрации данных, а также локальное приложение products для работы с товарами.

Настройки REST Framework включают пагинацию с размером страницы 12 элементов, что обеспечивает оптимальную загрузку данных. Также настроены backend-фильтры для поддержки фильтрации, поиска и сортировки через DjangoFilterBackend, SearchFilter и OrderingFilter.

CORS настроен для разрешения запросов с фронтенд-приложения, работающего на localhost:3000, что необходимо для взаимодействия React с Django API в процессе разработки.

Маршрутизация URL организована через роутер Django REST Framework, который автоматически создает endpoints для всех ViewSet'ов. Главный urls.py включает маршруты API через префикс /api/, а также настроено обслуживание медиа-файлов для изображений товаров в режиме разработки.

Команды управления Django используются для создания миграций базы данных и применения изменений схемы. Это обеспечивает версионирование структуры базы данных и упрощает развертывание на различных окружениях.

3.2 Реализация моделей данных

Модели данных представляют структуру информации в системе и реализованы в файле models.py приложения products.

Модель AnimalType (Тип животного) содержит поля для хранения названия типа животного, URL-идентификатора (slug) для использования в адресах, и опционального поля для иконки. Реализован метод сохранения, автоматически генерирующий slug из названия при создании записи.

Модель Brand (Производитель) включает информацию о названии бренда, стране производства, описании, логотипе и веб-сайте. Это позволяет предоставить пользователям детальную информацию о производителях товаров.

Модель Category (Категория) реализует иерархическую структуру категорий через самосвязь (self-referencing foreign key). Поле parent указывает на родительскую категорию, что позволяет создавать многоуровневую структуру,

например: Корма → Корма для собак → Сухие корма для собак. Категория также содержит поля для описания и изображения.

Модель Product (Товар) является центральной моделью системы и содержит всю информацию о товаре. Основные поля включают название, описание, состав, вес и цену. Связи с другими моделями реализованы через ForeignKey: связь с категорией, производителем и типом животного. Дополнительные поля включают возрастную группу (с предопределенными вариантами через choices), булевые флаги для особых характеристик (гипоаллергенный, беззерновой) и статус наличия товара.

Для оптимизации запросов к базе данных созданы индексы на часто используемых полях: составной индекс на category и animal_type для фильтрации, индекс на price для сортировки по цене, и индекс на created_at для отображения новых товаров.

Модель ProductImage (Изображения товара) позволяет добавлять несколько дополнительных изображений к товару, что важно для детального представления продукции. Поле order определяет порядок отображения изображений в галерее.

Все модели используют verbose_name для корректного отображения в админ-панели Django на русском языке.

3.3 Разработка REST API

REST API разработан с использованием Django REST Framework и обеспечивает доступ к данным для фронтенд-приложения.

Сериализаторы преобразуют модели Django в JSON-формат и обратно. Реализовано несколько типов сериализаторов:

– AnimalTypeSerializer, BrandSerializer и CategorySerializer предоставляют базовую информацию о соответствующих сущностях. CategorySerializer дополнительно включает название родительской категории через source='parent.name' для удобства отображения иерархии.

– ProductListSerializer используется для отображения списка товаров и содержит краткую информацию: название, изображение, цену, вес и связанные данные (название бренда, категории и типа животного). Использование source в полях serializer позволяет получать данные из связанных моделей без дополнительных запросов.

– ProductDetailSerializer предоставляет полную информацию о товаре, включая вложенные сериализаторы для бренда, категории и типа животного. Также включается список дополнительных изображений через related_name='additional_images'.

ViewSet'ы определяют логику обработки API-запросов:

– ProductViewSet наследуется от ReadOnlyModelViewSet, что автоматически создает endpoints для получения списка (list) и деталей (retrieve) товаров. Метод get_serializer_class динамически выбирает сериализатор в зависимости от действия: ProductDetailSerializer для детального просмотра и ProductListSerializer для списка.

Использование select_related и prefetch_related в queryset оптимизирует запросы к базе данных, предзагружая связанные объекты и избегая проблемы N+1 запросов.

Фильтрация реализована через класс ProductFilter, наследуемый от django_filters.FilterSet. Определены фильтры для диапазона цен (price_min и price_max), выбора бренда, категории, типа животного, возрастной группы и булевых характеристик.

Поиск настроен через search_fields, позволяя искать товары по названию, описанию и составу. Django REST Framework автоматически применяет полнотекстовый поиск к указанным полям.

Сортировка определена через ordering_fields, позволяя сортировать товары по цене, дате добавления и названию. Параметр ordering задает сортировку по умолчанию (новые товары первыми).

Пагинация настроена глобально в settings.py с размером страницы 12 элементов. Django REST Framework автоматически добавляет параметры page, next и previous в ответы API.

3.4 Разработка frontend на React

Frontend приложение построено на React 18 с использованием функциональных компонентов и хуков.

Структура приложения организована по компонентному принципу. Главный компонент App.js использует React Router для маршрутизации между страницами. Определены маршруты для главной страницы (HomePage), каталога (CatalogPage) и страницы товара (ProductDetailPage).

Компонент Header реализует шапку сайта с логотипом, формой поиска и навигацией. Поиск обрабатывается через контролируемый компонент (controlled component) с использованием useState хука. При отправке формы происходит навигация на страницу каталога с параметром search в URL.

Компонент ProductCard отображает карточку товара в виде ссылки на детальную страницу. Использует props для получения данных о товаре и формирует URL изображения, добавляя базовый адрес backend сервера. Условный рендеринг применяется для отображения бейджа "Нет в наличии" и тегов специальных характеристик.

Компонент Filters реализует панель фильтрации с множественными параметрами. Использует локальное состояние для хранения выбранных фильтров и передает изменения родительскому компоненту через колбэк onFilterChange. Загрузка данных для выпадающих списков (бренды, категории, типы животных) выполняется в useEffect при монтировании компонента.

Страница HomePage отображает приветственный баннер, сетку категорий и список новых товаров. Использует Promise.all для параллельной загрузки данных из нескольких API endpoints, что оптимизирует время загрузки страницы.

Страница CatalogPage реализует основной функционал каталога. Компонент использует useSearchParams хук для работы с параметрами URL, что позволяет сохранять состояние фильтров в адресной строке. При изменении фильтров или номера страницы отправляется новый запрос к API. Пагинация реализована с кнопками навигации и номерами страниц.

Страница ProductDetailPage использует useParams для получения slug товара из URL. Компонент отображает детальную информацию, включая галерею изображений с возможностью переключения между фотографиями. Хлебные крошки (breadcrumbs) обеспечивают удобную навигацию по уровням категорий.

Сервис API инкапсулирует взаимодействие с backend через axios. Определены функции для получения товаров, категорий, брендов и типов животных. Использование axios.create создает настроенный экземпляр с базовым URL и заголовками.

Стилизация выполнена с использованием CSS3 с модульным подходом – каждый компонент имеет свой CSS файл. Применены CSS Grid и Flexbox для создания адаптивной сетки. Media queries обеспечивают корректное отображение на различных устройствах.

3.5 Реализация системы поиска и фильтрации

Система поиска и фильтрации является ключевой функциональностью каталога, позволяющей пользователям быстро находить необходимые товары.

Полнотекстовый поиск реализован на уровне Django через SearchFilter из Django REST Framework. При получении параметра search в запросе выполняется поиск по полям name, description и composition модели Product. Django автоматически применяет SQL оператор ILIKE (регистронезависимый поиск) для каждого поля.

Фильтрация по диапазону цен реализована через два параметра: price_min и price_max. На backend используются NumberFilter с lookup_expr='gte' и 'lte' соответственно, что генерирует SQL условия WHERE price >= ? AND price <= ?.

На frontend пользователь вводит значения в два числовых поля, которые передаются как параметры запроса.

Фильтрация по категориям, брендам и типам животных выполняется через выпадающие списки (select). Значения загружаются при инициализации компонента Filters через отдельные API запросы. При выборе значения передается ID объекта, и Django Filter применяет фильтрацию через ForeignKey отношения.

Фильтрация по возрастной группе использует предопределенный набор значений из choices в модели Product. Frontend отображает все доступные варианты в select элементе, и выбранное значение передается как параметр запроса.

Булевые фильтры (гипоаллергенный, беззерновой) реализованы через чекбоксы. Значение передается как true/false, и Django BooleanFilter применяет точное совпадение в базе данных.

Комбинирование фильтров реализовано через объединение всех параметров в одном запросе. Django Q-объекты автоматически создают SQL запрос с множественными условиями через оператор AND. Это позволяет пользователям применять несколько фильтров одновременно для точного поиска.

Сброс фильтров выполняется через кнопку "Сбросить", которая устанавливает все параметры в начальные значения и отправляет новый запрос без фильтров.

Сортировка результатов поддерживается через параметр ordering. Пользователь может сортировать товары по цене (возрастанию и убыванию), дате добавления и названию. На backend OrderingFilter автоматически применяет сортировку через SQL ORDER BY.

Пагинация разделяет большой список товаров на страницы по 12 элементов. Django REST Framework возвращает в ответе количество результатов (count), URL следующей и предыдущей страниц. Frontend вычисляет общее количество страниц и отображает навигацию.

Оптимизация производительности достигается через несколько подходов. Использование select_related и prefetch_related минимизирует количество запросов к базе данных. Индексы на часто фильтруемых полях ускоряют выполнение SQL запросов. Пагинация ограничивает объем передаваемых данных.

Пользовательский опыт улучшается через сохранение состояния фильтров в URL параметрах. Это позволяет пользователям делиться ссылками на отфильтрованные результаты и использовать навигацию браузера (кнопки назад/вперед) без потери состояния фильтров.

3.6 Интеграция с системой учета 1С Предприятие

Одним из важных практических аспектов проекта является интеграция с системой учета 1С: Предприятие 8.3 для Беларуси, которая используется зоомагазином для ведения складского и торгового учета. Интеграция позволяет автоматизировать процесс обновления каталога и обеспечить актуальность информации о товарах, ценах и остатках.

Архитектура интеграции основана на периодической синхронизации данных через промежуточные файлы Excel. Выбран подход выгрузки данных из 1С в файлы с последующим импортом в Django, что не требует прямого подключения к базе данных 1С и обеспечивает простоту реализации.

Процесс интеграции разделен на несколько этапов:

– Первичная загрузка номенклатуры выполняется из файла, содержащего полный справочник товаров. Файл включает артикулы, наименования, полные наименования (используемые как описания), марки производителей и виды номенклатуры. Разработана команда управления Django import_from_1c, которая обрабатывает файл Excel и создает записи товаров в базе данных.

– Команда автоматически распознает структуру файла, определяя колонки по их названиям. Поддерживаются различные варианты названий колонок (например, "Наименование", "Название", "Товар" для названия товара). Это обеспечивает гибкость при изменении формата выгрузки из 1С.

– Обработка многоуровневой структуры категорий реализована через анализ колонки "Вид номенклатуры" или аналогичных полей. При импорте автоматически создаются записи категорий и устанавливаются связи между товарами и категориями. Если категория уже существует, используется существующая запись.

– Автоматическое создание производителей выполняется аналогично категориям. При обнаружении нового бренда в колонке "Марка (Бренд)" создается запись производителя. Для товаров без указания производителя используется значение по умолчанию.

– Генерация URL-идентификаторов (slug) для товаров выполняется автоматически на основе артикула (если присутствует) или названия товара. Функция slugify преобразует кириллические символы в транслит и обеспечивает уникальность идентификаторов.

– Синхронизация цен и остатков осуществляется через отдельный отчет из 1С, содержащий информацию об остатках на складе и розничных ценах. Разработана команда sync_stock_prices для обработки этого отчета.

– Команда анализирует файл с остатками и ценами, извлекая данные из колонок "Остаток на складе" и "Стоимость (в розничных ценах)". Для каждой строки выполняется поиск соответствующего товара в базе данных по названию.

– Алгоритм сопоставления товаров работает в несколько этапов. Сначала выполняется поиск по точному совпадению названия. Если точное совпадение не найдено, выполняется поиск по первым словам названия. Наконец, применяется поиск по частичному совпадению. Такой подход обеспечивает высокую вероятность нахождения товара даже при небольших различиях в написании.

– Обновление цен выполняется путем замены значения в поле price модели Product. Цены извлекаются из колонки с розничной стоимостью и преобразуются в тип Decimal для точных финансовых расчетов.

– Определение статуса наличия основано на значении остатка. Если остаток больше нуля, товару присваивается статус "В наличии" (in_stock). При нулевом остатке устанавливается статус "Нет в наличии" (out_of_stock). Это позволяет пользователям видеть актуальную информацию о доступности товаров.

Команды управления реализованы с использованием подсистемы Django management commands, что обеспечивает их запуск из командной строки с различными параметрами. Команда import_from_1c принимает путь к файлу Excel и опциональный флаг --update для обновления существующих записей. Команда sync_stock_prices также принимает путь к файлу и поддерживает режим предпросмотра --dry-run для проверки изменений без их применения. Обработка ошибок и статистика включены в обе команды. В процессе выполнения выводится детальная информация о каждом обработанном товаре. По завершении отображается итоговая статистика: количество созданных, обновленных и пропущенных записей, а также количество товаров, которые не удалось найти в базе данных. Практическое применение интеграции заключается в следующем процессе. Первоначально выполняется импорт полной номенклатуры из 1С командой import_from_1c. Затем регулярно (ежедневно или еженедельно) выполняется синхронизация цен и остатков командой sync_stock_prices. При появлении новых товаров в 1С повторно запускается импорт номенклатуры с флагом обновления.

Преимущества реализованной интеграции включают автоматизацию обновления каталога, что устраняет необходимость ручного ввода данных; актуальность информации о ценах и остатках; простоту использования через командную строку; гибкость настройки через параметры команд; детальное логирование процесса для контроля и отладки.

3.7 Контейнеризация приложения

Контейнеризация приложения с использованием Docker обеспечивает упрощение развертывания системы. Созданы Dockerfile для backend и frontend компонентов, а также docker-compose.yml для оркестрации всех сервисов. Docker Compose автоматически запускает PostgreSQL, Django и React в отдельных контейнерах с настроенным взаимодействием между ними. Это позволяет развернуть полный стек приложения одной командой docker-compose up, что значительно упрощает установку на новых окружениях.

.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы был разработан полнофункциональный веб-каталог зоомагазина с системой поиска и фильтрации товаров.

Достигнуты все поставленные цели и решены задачи проекта. Проведен анализ существующих решений в области каталогов зоотоваров, выявлены их недостатки и определены требования к разрабатываемой системе.

Обоснован выбор технологического стека Django + React + PostgreSQL как оптимального для реализации поставленных задач. Спроектирована архитектура веб-приложения с разделением на frontend и backend компоненты.

Разработана структура базы данных, включающая пять взаимосвязанных таблиц с поддержкой иерархических категорий и множественных изображений товаров.

Создана схема REST API для взаимодействия между компонентами системы. Реализован backend на Django с использованием Django REST Framework. Созданы модели данных для товаров, категорий, производителей и типов животных. Настроена панель администрирования для удобного управления каталогом.

Реализована комплексная система поиска и фильтрации товаров по множеству параметров: цене, производителю, категории, типу животного, возрастной группе и специальным характеристикам. Система поддерживает полнотекстовый поиск, комбинирование фильтров, сортировку и пагинацию результатов.

Реализована интеграция с системой учета 1С Предприятие 8.3, что позволяет автоматически импортировать номенклатуру, цены и остатки товаров. Создан механизм синхронизации данных через файлы Excel, выгруженные из 1С.

Практическая значимость работы подтверждается созданием полнофункционального веб-каталога с реальными данными из учетной системы 1С: Предприятие.

Разработанная система может быть расширена дополнительным функционалом:

– углубление интеграции с системой 1С через веб-сервисы для автоматической синхронизации в реальном времени;

– реализация системы отзывов и рейтингов товаров;

– добавление системы рекомендаций на основе просмотренных товаров;

– внедрение системы уведомлений о поступлении товаров;

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Django Documentation [Электронный ресурс]. – Режим доступа: https://docs.djangoproject.com/. – Дата доступа: 9.11.2025.

2. React Documentation [Электронный ресурс]. – Режим доступа: https://react.dev/. – Дата доступа: 20.11.2025.

3. Django REST Framework [Электронный ресурс]. – Режим доступа: https://www.django-rest-framework.org/.

4. Мэтиз, Э. Изучаем Python. Программирование игр, визуализация данных, веб-приложения / Э. Мэтиз. – СПб. : Питер, 2020. – 496 с.

5. Дронов, В. А. Django 4. Подробное руководство для разработчиков / В. А. Дронов. – СПб. : БХВ-Петербург, 2023. – 912 с.

6. Бэнкс, А. React и Redux: функциональная веб-разработка / А. Бэнкс, Е. Порселло. – СПб. : Питер, 2022. – 336 с.

7. PostgreSQL Documentation [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs/. – Дата доступа: 9.11.2025.

8. MDN Web Docs: JavaScript [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/JavaScript. – Дата доступа: 28.11.2025.

9. React Router Documentation [Электронный ресурс]. – Режим доступа: https://reactrouter.com/. – Дата доступа: 10.12.2025.

11. Фримен, Э. Изучаем программирование на HTML5 / Э. Фримен, Э. Робсон. – СПб. : Питер, 2021. – 768 с.

12. Фримен, А. Pro React 16 / А. Фримен. – Apress, 2019. – 693 p.

16. Stack Overflow [Электронный ресурс]. – Режим доступа: https://stackoverflow.com/. – Дата доступа: 9.11.2025.

19. Web Content Accessibility Guidelines (WCAG) [Электронный ресурс]. – Режим доступа: https://www.w3.org/WAI/standards-guidelines/wcag/.

20. REST API Tutorial [Электронный ресурс]. – Режим доступа: https://restfulapi.net/.